

- Contenedores
- AWS y contenedores
 - Cómo funciona Fargate
- AWS CLI y credenciales (para labs)
- Subida de imágenes
 - OPCIÓN 1: Subir imagen por **AWS CLI**
 - OPCIÓN 2: Subir imagen por **interfaz gráfica (GUI de AWS)**
 - En resumen
- Repositorio en ECR (Elastic Container Service)
 - Ya tienes tu imagen subida a **ECR**, así que el siguiente paso es **lanzar el contenedor con AWS Fargate (ECS)**.
- Lanzar el contenedor con AWS Fargate (ECS)
 - Resumen visual
- Explicación de recursos
- Afianzando Conceptos
 - Cuando trabajas dentro de **AWS Academy**, **NO estás en tu cuenta personal de AWS**, sino en una **cuenta temporal** gestionada por el entorno educativo.
 - Aunque no puedes conservar el cluster en AWS Academy, sí puedes **guardar tu trabajo localmente**:
- Docker Compose
 - Ejemplo local: Flask + Postgres con Docker Compose
 - Llevar **Docker Compose** a AWS Fargate
- En resumen
- Kubernetes
 - Kubernetes en la nube
- Diferencia entre Docker, Docker Compose, Fargate y Kubernetes
- PROBLEMAS ENCONTRADOS
 - Entendiendo la limitación: por qué **NO podemos usar ECS ni Fargate en AWS Academy**
 - Simulación de despliegue Fargate
 - Alternativa: prácticas con **EC2 + Docker**
- Conclusión didáctica
- EJERCICIO A ENTREGAR
- Entrega:

Contenedores

- Qué son: Entornos ligeros que empaquetan tu aplicación con todas sus dependencias.
- Ventajas: Portabilidad, rápido arranque, menor consumo de recursos que una VM.
- Algunos conceptos clave:
 - Dockerfile: Instrucciones para crear la imagen.
 - Imagen: Plantilla lista para ejecutar.
 - Contenedor: Instancia en ejecución de la imagen.

AWS y contenedores

- ECR (Elastic Container Registry): Repositorio privado para guardar imágenes Docker en AWS.
- ECS (Elastic Container Service): Orquestador de contenedores en AWS.
 - Es el **servicio de orquestación** de contenedores de AWS.
 - Se encarga de **gestionar, escalar y coordinar** tus contenedores (dónde corren, cuántos hay, reinicios, balanceo, etc.).
 - **ECS no ejecuta contenedores directamente**, sino que necesita un **“motor de ejecución” (launch type)**:
 - **EC2** → ejecutas los contenedores en instancias EC2 (tú gestionas los servidores).
 - **Fargate** → ejecutas los contenedores sin servidores (AWS gestiona la infraestructura).
- Fargate: Ejecuta contenedores sin necesidad de servidores (“serverless”).
 - Es un **modo de ejecución de ECS** (un “launch type”).
 - Permite **ejecutar los contenedores sin tener que crear ni administrar servidores EC2**.
 - AWS se encarga de todo el aprovisionamiento, escalado y mantenimiento de la infraestructura.
 - Tú solo defines la imagen, CPU, memoria, red y puertos.

AWS Fargate es un servicio de ejecución de contenedores serverless Solo subes tu contenedor (Docker), defines la cantidad de recursos (CPU y memoria), y Fargate se encarga de ejecutarlo.

Cómo funciona Fargate

- Creas una imagen de contenedor (por ejemplo, con Docker).
- La subes a Amazon ECR (Elastic Container Registry) o cualquier otro registro de contenedores.
- Creas una Task Definition en ECS que describe tu contenedor:
 - Imagen
 - Puerto
 - Variables de entorno
 - Recursos (CPU y memoria)
- Lanzas un servicio o tarea con Fargate:
 - ECS se encarga de crear instancias virtuales, asignar red, balanceadores, etc.
- Tu contenedor se ejecuta en la nube, y solo pagas por los recursos que consume mientras está activo.

AWS CLI y credenciales (para labs)

- AWS CLI permite controlar AWS desde la terminal.
 - Si quieres instalar AWS CLI en Linux

```
sudo apt update
sudo apt install awscli -y
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
sudo apt update
sudo apt install unzip -y
unzip awscliv2.zip
sudo ./install
aws --version
```

- Si quieres instalar AWS CLI en Windows. Descargar el instalador
 - Ve a la página oficial de AWS CLI: AWS CLI v2 Windows Installer, O directamente: awscli-exe-windows-x86_64.msi
 - Ejecutar el instalador Haz doble clic en el archivo .msi que descargaste. Sigue el asistente de instalación. Acepta las opciones por defecto (se instalará en C:\Program Files\Amazon\AWSCLIV2).

Ya tienes instalado AWS CLI en tu máquina local, fuera del laboratorio. Ahora toca configurarlo para que se conecte con el laboratorio.

A tener en cuenta En entornos reales con hacer la configuración una sola vez es suficiente. Al trabajar con cuentas académicas, toca realizar esta configuración cada vez que arrancamos el laboratorio ya que las credenciales de configuración son temporales.

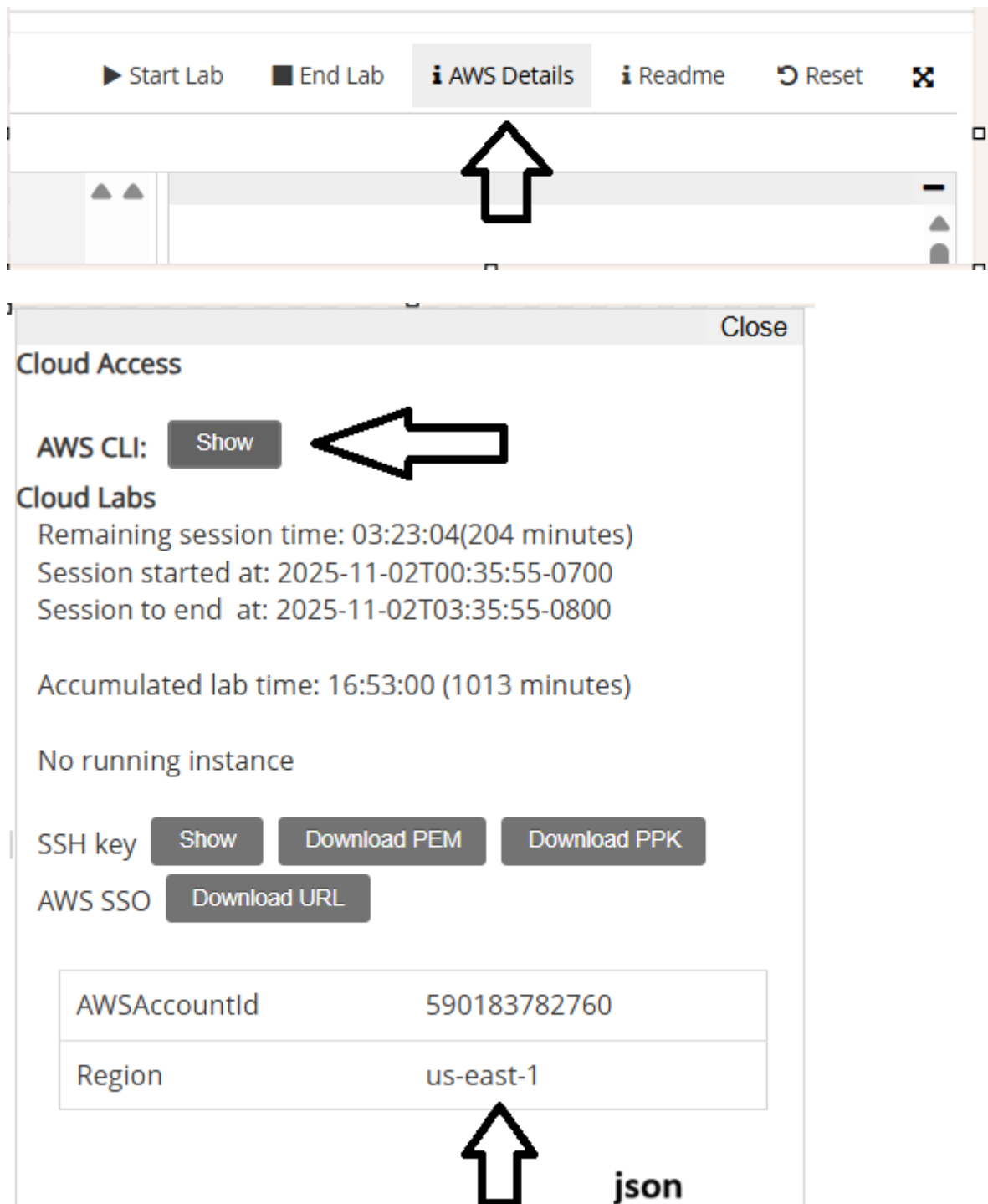
- Configuración básica en laboratorio AWS (Academy) / AWS:

```
aws configure
```

```
PS C:\Users\santi> aws configure
AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
AWS Session Token [None]: 
Default region name [None]: us-east-1
Default output format [None]: json
```

- Donde buscamos la información:

- Si estas en entorno real estos datos se encuentran en la página de inicio de AWS.
- Si estas en AWS Academy, estos datos son temporales y se encuentran en



recuerda que estas credenciales son temporales en entornos académicos

- Hemos configurado:

```
AWS Access Key ID:    ASIAxxxxEXAMPLE
AWS Secret Access Key: xxxxEXAMPLEKEY
AWS Session Token:    xxxx
Default region name:   xxx
Default output format: json
```

- Verificación:

```
aws sts get-caller-identity
```

```
PS C:\Users\santi> aws sts get-caller-identity
{
  "UserId": [REDACTED],
  "Account": [REDACTED],
  "Arn": [REDACTED]
}
```

Ya esta configurado nuestro laboratorio

Subida de imágenes

Flujo de trabajo típico

- **La imagen**
 - Crear aplicación (por ejemplo, Flask + Python).
 - Crear Dockerfile y construir la imagen:
 - `docker build -t mi-app .`
 - Probar localmente con Docker o Docker Compose.
- **Subir la imagen a ECR:** Dos métodos completos: **1. Por consola (CLI)** **2. Por la interfaz web (GUI)**

OPCIÓN 1: Subir imagen por **AWS CLI**

Esta es la forma más común y profesional, y te servirá en cualquier entorno (laboratorio o cuenta propia).

Supongamos que ya tienes una imagen local llamada `flask-app:latest`.

Paso 1: Crear un repositorio en ECR (Elastic Container Repository)

```
aws ecr create-repository --repository-name flask-app
```

Verás una salida parecida a:

```
{
  "repository": {
    "repositoryUri": "590183782760.dkr.ecr.us-east-1.amazonaws.com/flask-app",
    "repositoryName": "flask-app",
    ...
  }
}
```

```
}  
}
```

Anota el **repositoryUri**, porque lo necesitarás.

Paso 2: Autenticación con Docker

Ejecuta este comando para hacer login en ECR (reemplaza la región si no es **us-east-1**):

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --  
password-stdin 590183782760.dkr.ecr.us-east-1.amazonaws.com
```

Cómo saber tu Account ID y región

- En tu panel de Cloud Access del laboratorio verás:

```
AWSAccountId    590183782760  
Region          us-east-1
```

Esos dos valores son los que debes usar para construir la URL de ECR:

```
<AccountId>.dkr.ecr.<region>.amazonaws.com
```

- Si todo está bien configurado, te dirá:

```
Login Succeeded
```

Paso 3: Etiquetar tu imagen local

Tu imagen local (por ejemplo **flask-app:latest**) debe tener la misma etiqueta que el repositorio ECR.

Ejemplo:

```
docker tag flask-app:latest 590183782760.dkr.ecr.us-east-1.amazonaws.com/flask-  
app:latest
```

Paso 4: Subir (push) la imagen

```
docker push 590183782760.dkr.ecr.us-east-1.amazonaws.com/flask-app:latest
```

Verás cómo Docker sube las capas una a una. Cuando termine, te mostrará algo como:

```
latest: digest: sha256:abcd1234... size: 1234
```

¡Imagen subida correctamente a ECR!

Paso 5: Verificar

Puedes listar tus repositorios o imágenes:

```
aws ecr describe-repositories
```

O ver las imágenes dentro de un repositorio:

```
aws ecr list-images --repository-name flask-app
```

OPCIÓN 2: Subir imagen por **interfaz gráfica (GUI de AWS)**

Esta forma es más visual y útil si estás empezando.

Paso 1: Ir a ECR

1. Abre la **consola AWS** (desde el botón AWS de tu laboratorio o desde la URL del lab).
2. En la barra de búsqueda, escribe **ECR** y entra a **Elastic Container Registry**.

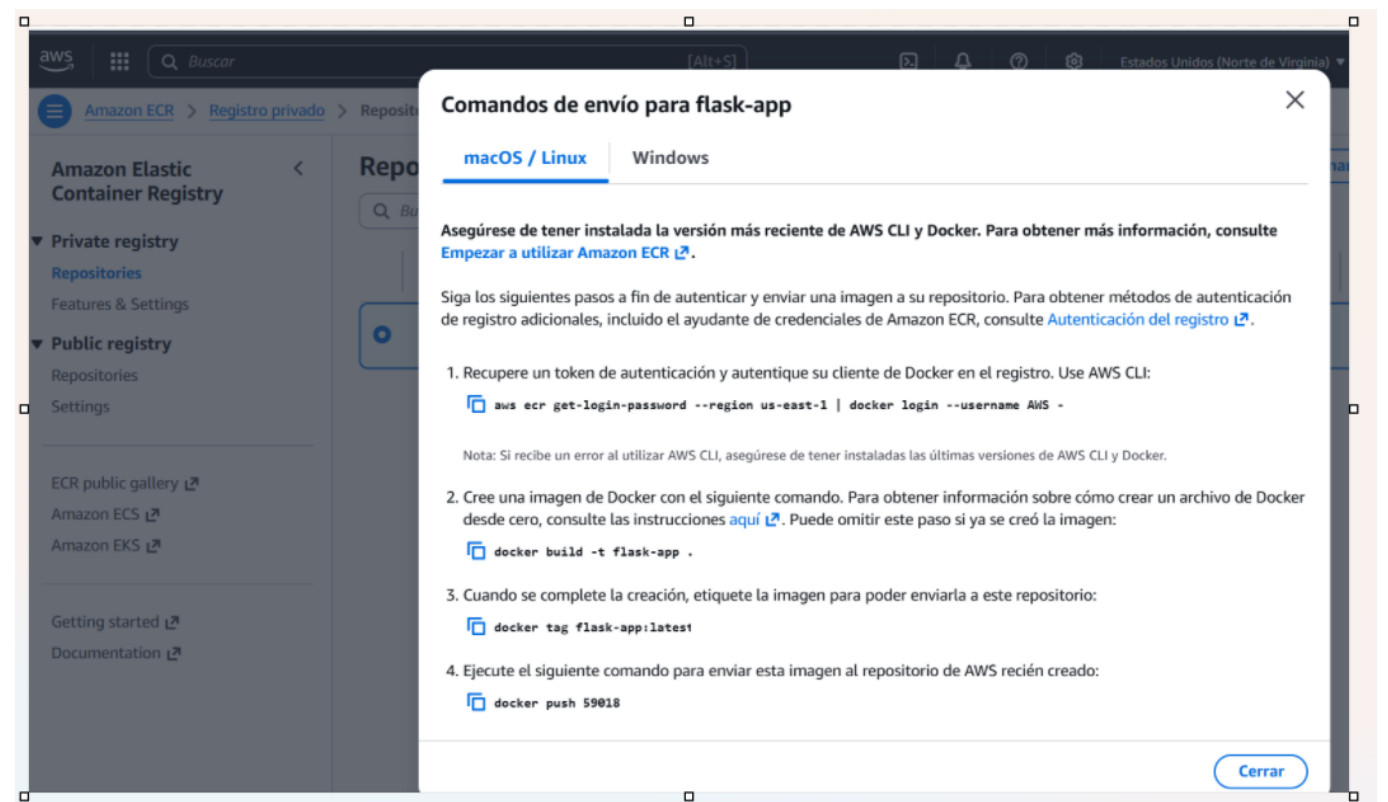
Paso 2: Crear un repositorio

1. Clic en **“Create repository”**
2. Nombre del repositorio → por ejemplo: **flask-app**
3. Deja todo por defecto y clic en **“Create repository”**

Paso 3: Obtener los comandos de subida

Después de crear el repositorio, verás una página con detalles y un botón que dice: **“View push commands”**

Haz clic ahí y AWS te mostrará **exactamente los comandos Docker** que necesitas para subir la imagen desde tu máquina local (serán casi idénticos a los de la opción A).

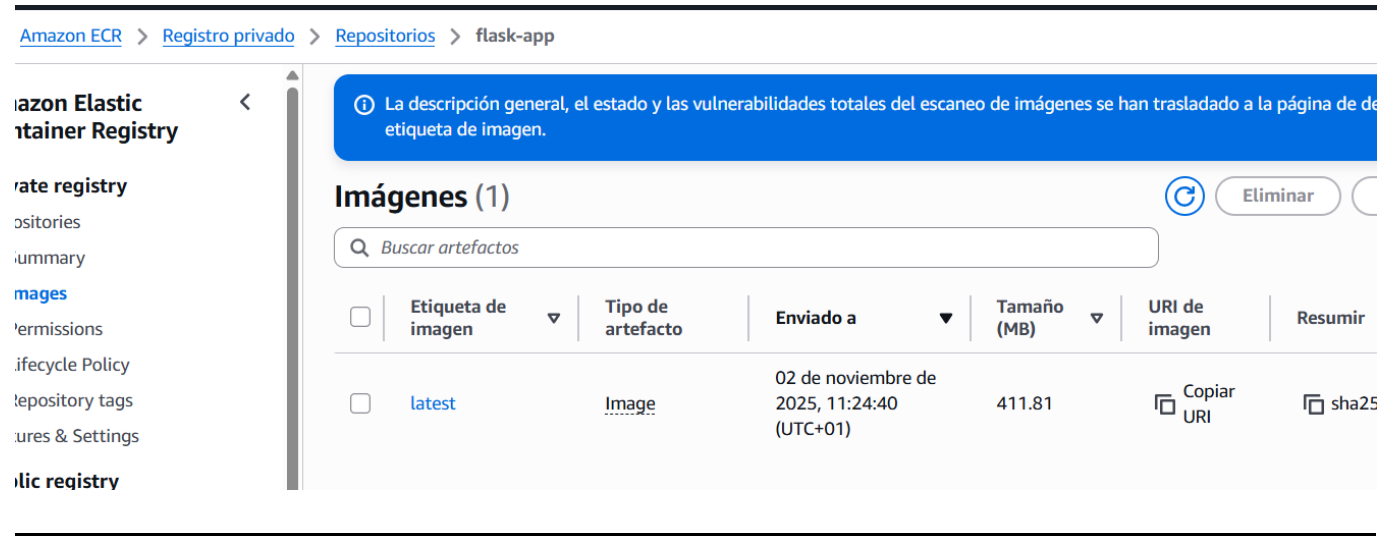


Cópalos y pégalos en tu terminal.

Paso 4: Verificar desde la GUI

- 1. En la consola de ECR, entra en tu repositorio `flask-app`.
- 2. Verás tu imagen bajo la pestaña “**Imágenes**”.
 - Aparecerá con su tag (`latest`)
 - Digest (`sha256:...`)
 - Tamaño y fecha de subida

¡Tu imagen ya está en AWS!



En resumen

Paso	CLI	GUI
Crear repositorio	<code>aws ecr create-repository</code>	Botón Create repository
Login Docker	<code>`aws ecr get-login-password ...`</code>	View push commands
Etiquetar imagen	<code>docker tag ...</code>	View push commands
Subir imagen	<code>docker push ...</code>	View push commands
Verificar subida	<code>aws ecr list-images</code>	Ver pestaña Images

Repositorio en ECR (Elastic Container Service)

Un **repositorio** en AWS ECR es un **contenedor lógico (una carpeta)** donde se guardan **imágenes Docker relacionadas**.

Qué guarda un repositorio

Guarda **una o varias imágenes**, normalmente distintas versiones de una misma app, diferenciadas por **etiquetas (tags)**.

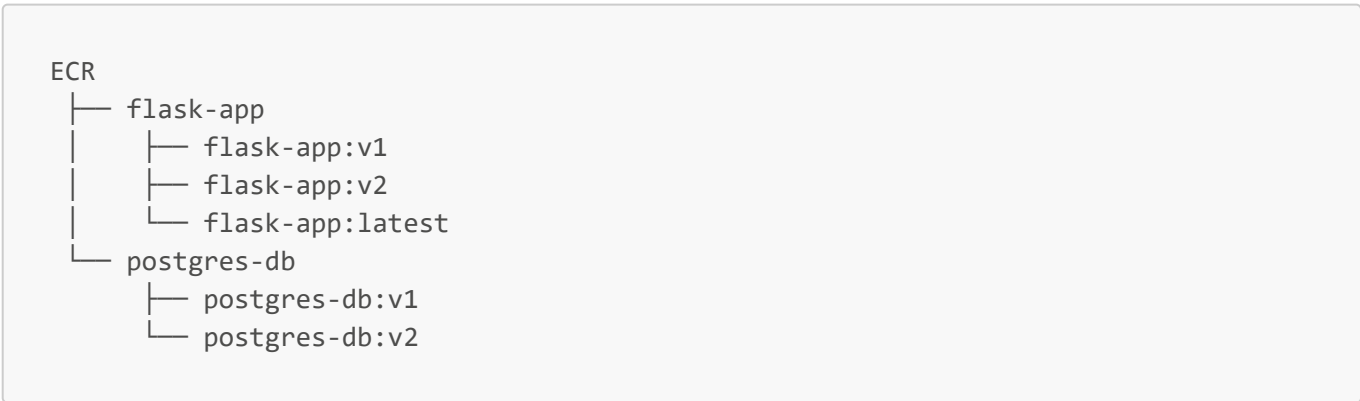
Por ejemplo, si tienes una app Flask:

Imagen	Etiqueta (tag)	Descripción
flask-app:latest	<code>latest</code>	Última versión
flask-app:v1	<code>v1</code>	Versión inicial
flask-app:v2	<code>v2</code>	Nueva versión con mejoras

Todas esas imágenes están **dentro del mismo repositorio** `flask-app`.

Cómo se organizan

Estructura típica de ECR:



Cada **repositorio** → un conjunto de **imágenes relacionadas**.

¿Cuántas imágenes puede tener?

Un solo repositorio puede contener **decenas o cientos de imágenes**, cada una con su **tag**.

No hay un límite fijo por número de imágenes, sino por **espacio total (almacenamiento)**, que en el laboratorio no suele ser un problema.

Qué pasa en AWS Academy (laboratorios)

Elemento	Qué pasa con él en el laboratorio
Repositorio (ECR)	Se crea dentro de tu cuenta temporal del laboratorio
Imágenes	Se guardan en ese repositorio
Duración	Solo mientras el laboratorio esté activo
Después del cierre	Se borra todo: repositorios, imágenes, roles, etc.

- El **repositorio y las imágenes sí se guardan en AWS**, pero **solo temporalmente**.
- Cuando se acaba el laboratorio, la **cuenta entera se destruye**, así que desaparece todo lo que creaste (ECR, ECS, EC2, etc.).

Ya tienes tu imagen subida a **ECR**, así que el siguiente paso es **lanzar el contenedor con AWS Fargate (ECS)**.

Lanzar el contenedor con AWS Fargate (ECS)

AWS **Fargate** es un servicio que **ejecuta contenedores sin servidores EC2**. Tú defines:

- Qué imagen usar (de ECR)
- Cuántos contenedores lanzar
- Cuánta CPU y memoria quieres
- En qué red (subred/VPC) correrá

Y **Fargate se encarga de todo lo demás** (no gestionas máquinas).

PASOS PARA LANZAR TU IMAGEN CON FARGATE (ECS, Elastic Container Service)

Paso 1.- Entra en la consola de AWS

- Desde tu laboratorio (AWS Academy → Cloud Access → "AWS").
- Busca el servicio **ECS (Elastic Container Service)**.

Paso 2.- Crea un **Cluster** nuevo

El **cluster** es el "grupo lógico" donde se ejecutan tus tareas.

- En el menú izquierdo → **Clusters** → **Create Cluster**
- Elige **Networking only (Fargate)**
- Ponle nombre, por ejemplo:

```
flask-cluster
```

- Deja las opciones por defecto y haz clic en **Create** ☒

Paso 3.- Crea una *Task Definition*

Esto es una **plantilla** que le dice a Fargate **qué contenedor lanzar** y cómo hacerlo.

1. En el menú izquierdo, ve a **Task Definitions** → **Create new Task Definition**
2. Elige **FARGATE**
3. Asigna un nombre:

```
flask-task
```

4. **Role:** selecciona el que viene por defecto del laboratorio → **LabRole** (si no aparece, créalo o usa el que tenga permisos básicos de ECS).
5. **Task memory y CPU:** elige algo sencillo:

```
Memory: 0.5 GB  
CPU: 0.25 vCPU
```

6. En la sección **Container definitions**, pulsa **Add container** y completa:

Campo	Valor
Container name	flask-app
Image	590183782760.dkr.ecr.us-east-1.amazonaws.com/flask-app:latest
Port mappings	5000 → 5000 (si tu app Flask usa ese puerto)

Guarda con **Add** → luego **Create**.

¡Ya tienes la *Task Definition*!

Paso 4.- Ejecuta la tarea (Task)

Ahora le dices a ECS "lanza un contenedor con esta definición".

1. Ve a **Clusters** → **flask-cluster**
2. Pulsa **Run new task**
3. Configura:

- **Launch type:** `FARGATE`
- **Task definition:** selecciona `flask-task:1`
- **Cluster:** `flask-cluster`
- **Number of tasks:** `1`
- **Platform version:** `LATEST`

4. En **Networking**, elige una VPC (por defecto suele haber una).

- Selecciona al menos **una Subnet**.
- Marca **Auto-assign public IP** → **ENABLED** (importante para acceder desde fuera).

Haz clic en **Run task**

Paso 5.-Verifica que está corriendo

- Ve al **Cluster** → **Tasks** → **flask-task**
- Estado: debería decir **RUNNING** 
- Haz clic sobre el **Task ID**
- En la pestaña **Details**, busca el **Public IP** asignado.

Si tu app Flask escucha en el puerto 5000, abre en tu navegador:

```
http://<PUBLIC-IP>:5000
```

Deberías ver tu aplicación corriendo en Fargate.

Paso 6.- (Opcional) Parar o eliminar

Cuando termines:

- Puedes **Stop task** desde el panel.
- O borrar el *cluster* si ya no lo necesitas.

Resumen visual

```
ECR → guarda tu imagen
ECS (con Fargate) → ejecuta esa imagen sin servidores
├── Cluster (grupo de tareas)
│   ├── Task Definition (plantilla de contenedor)
│   │   └── Container (imagen, CPU, memoria, puertos)
└── Task (instancia ejecutándose)
```

Explicación de recursos

Concepto	Qué es	Ejemplo
ECR	Repositorio de imágenes	<code>flask-app:latest</code>
Task Definition	Plantilla que define cómo ejecutar el contenedor	CPU, memoria, imagen
Cluster	Agrupación lógica de tareas	<code>flask-cluster</code>
Task	Ejecución individual de la plantilla	Contenedor Flask en marcha
Fargate	Motor de ejecución sin servidores	No hay EC2, AWS lo gestiona

Afianzando Conceptos

Cuando trabajas dentro de **AWS Academy**, **NO estás en tu cuenta personal de AWS**, sino en una **cuenta temporal** gestionada por el entorno educativo.

- Todo lo que creas (ECR, ECS, Cluster, Roles, etc.) existe solo dentro de **esa sesión de laboratorio**
- Cada vez que inicias el laboratorio, AWS Academy te crea **una cuenta temporal de AWS** con su propio **Account ID** (por ejemplo: `590183782760`).
- Cualquier recurso que crees (repositorios ECR, clusters ECS, instancias, roles IAM, etc.) **se guarda dentro de esa cuenta temporal**.

-
- Cuando el laboratorio expira o lo cierras... **Esa cuenta se destruye completamente.**
 - Se eliminan automáticamente:
 - Tu **Cluster de ECS (Fargate)**
 - Tus **Tareas (Tasks)**
 - Tus **Task Definitions**
 - Tus **Repositorios ECR**
 - Tus **Imágenes Docker**
 - Cualquier **VPC, Subnet, Security Group, o Role** creado dentro

todo desaparece.

- Cuánto dura

Depende del tiempo de práctica que te dé el instructor o el módulo:

- Normalmente tienes entre **2 y 4 horas de sesión activa**.
- Cuando el contador llega a 0 o pulsas **"End Lab"**, se borra la cuenta entera.
- Si inicias un nuevo laboratorio más tarde, obtendrás:

- Una **cuenta nueva**
- Un **nuevo Account ID**
- Nuevas **credenciales**
- Todo vacío (como si empezaras desde cero)

- Qué puedes hacer para **no perder el trabajo**

Aunque no puedes conservar el cluster en AWS Academy, sí puedes **guardar tu trabajo localmente**:

A. Guarda tus archivos de configuración

Antes de cerrar el lab:

- Descarga o guarda en tu máquina local:
 - `Dockerfile`
 - `docker-compose.yml`
 - Código fuente de tu app (por ejemplo, `app.py`)

B. Guarda tu imagen localmente

Antes de que expire el lab, haz:

```
docker pull 590183782760.dkr.ecr.us-east-1.amazonaws.com/flask-app:latest
docker save -o flask-app.tar flask-app:latest
```

Así tendrás un archivo `flask-app.tar` que podrás **volver a cargar** en otro laboratorio o en tu propia cuenta AWS personal con:

```
docker load -i flask-app.tar
```

Elemento	Qué pasa al cerrar el lab	Cómo conservarlo
Cluster ECS	Se borra	No se puede conservar
ECR (repositorio e imágenes)	Se borra	Guarda la imagen con <code>docker save</code>
Roles / IAM	Se borran	N/A
Código fuente	> Si lo tienes local, lo conservas	Guarda tus ficheros en tu PC
Dockerfile / Config	Local	Guarda todo en tu proyecto local

El **cluster de Fargate** que creas en AWS Academy existe solo durante la sesión del laboratorio.

Una vez terminas o expira, **se elimina completamente** junto con todos los recursos asociados.

Solo lo que hayas **guardado localmente** (código, imágenes exportadas, etc.) permanece.

Docker Compose

Es una forma más simple y cómoda de definir y ejecutar tu aplicación con varios contenedores (por ejemplo, tu app Flask + una base de datos Postgres).

En AWS puedes usar **Docker Compose** de dos formas:

- **Localmente**, para probar tu app.
- **En AWS ECS con Fargate**

Ejemplo local: Flask + Postgres con Docker Compose

Estructura de tu proyecto

```
flask-app/  
├─ app.py  
├─ requirements.txt  
├─ Dockerfile  
└─ docker-compose.yml
```

Ejecutar localmente

Desde la carpeta del proyecto:

```
docker compose up --build
```

Luego abre en el navegador: <http://localhost:5000>

Verás tu aplicación Flask corriendo y conectándose al contenedor Postgres.

Llevar **Docker Compose** a AWS Fargate

- Paso 1. Requisitos previos
 - Tienes un archivo `docker-compose.yml` funcionando localmente.
 - Tienes configurada la **AWS CLI**:

```
aws configure
```

(usa las credenciales y región de tu laboratorio o cuenta).

Si estás en **AWS Academy**, asegúrate de usar las credenciales temporales actuales.

-
- Paso 2. Crear un repositorio ECR (para tus imágenes Docker)

Amazon ECS necesita que las imágenes estén almacenadas en **ECR (Elastic Container Registry)**.

```
aws ecr create-repository --repository-name mi-app
```

Guarda el valor del **repositoryUri** que aparece en la respuesta, algo así como:

```
<aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/mi-app
```

-
- Paso 3. Construir y subir tus imágenes a ECR

- 3.1. Inicia sesión en ECR:

```
aws ecr get-login-password --region us-east-1 | docker login --username  
AWS --password-stdin <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com
```

- 3.2. Construye la imagen desde tu proyecto (si usas docker-compose, puedes construir una imagen general):

```
docker compose build
```

- 3.3 Etiqueta tu imagen:

```
docker tag mi-app:latest <aws_account_id>.dkr.ecr.us-east-  
1.amazonaws.com/mi-app:latest
```

- 3.4. Súbela a ECR:

```
docker push <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/mi-  
app:latest
```

-
- Paso 4. Crear un **Cluster ECS**

```
aws ecs create-cluster --cluster-name mi-cluster
```


- Paso 5. Definir una **Task Definition** (plantilla de contenedor)

Crea un archivo JSON, por ejemplo `task-def.json`:

```
{
  "family": "mi-app-task",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "containerDefinitions": [
    {
      "name": "mi-app",
      "image": "<aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/mi-app:latest",
      "portMappings": [
        {
          "containerPort": 80,
          "protocol": "tcp"
        }
      ],
      "essential": true
    }
  ],
  "executionRoleArn": "arn:aws:iam::<aws_account_id>:role/ecsTaskExecutionRole"
}
```

Luego ejecútalo:

```
aws ecs register-task-definition --cli-input-json file://task-def.json
```

-
- Paso 6. Crear un **Servicio ECS Fargate**

Este comando lanza tu contenedor en Fargate:

```
aws ecs create-service \
  --cluster mi-cluster \
  --service-name mi-app-service \
  --task-definition mi-app-task \
  --launch-type FARGATE \
  --desired-count 1 \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-XXXXX],securityGroups=[sg-XXXXX],assignPublicIp=ENABLED}"
```

Reemplaza `subnet-XXXXX` y `sg-XXXXX` por los de tu VPC.

- Paso 7. Verificar que tu servicio está corriendo

```
aws ecs list-services --cluster mi-cluster
aws ecs describe-services --cluster mi-cluster --services mi-app-service
```

También puedes abrir la consola de **ECS** → **Clusters** → **mi-cluster** → **Services** para verlo gráficamente.

- Paso 8. Limpiar (opcional)

Si quieres borrar todo cuando termines:

```
aws ecs delete-service --cluster mi-cluster --service mi-app-service --force
aws ecs delete-cluster --cluster mi-cluster
aws ecr delete-repository --repository-name mi-app --force
```

En resumen

Objetivo	Comando principal
Crear repositorio ECR	<code>aws ecr create-repository</code>
Subir imagen Docker	<code>docker push <repo></code>
Crear cluster ECS	<code>aws ecs create-cluster</code>
Registrar tarea	<code>aws ecs register-task-definition</code>
Crear servicio Fargate	<code>aws ecs create-service</code>

Kubernetes

Kubernetes (también llamado **K8s**) es una **plataforma de orquestación de contenedores**. Su función principal es **gestionar, escalar y mantener contenedores** (por ejemplo, de Docker) **de forma automática**.

En palabras simples:

Si Docker crea y ejecuta un contenedor, Kubernetes se encarga de **controlar cientos o miles de contenedores** a la vez.

Imagina que tienes una aplicación web (por ejemplo, Flask) que usa varios contenedores:

- 1 contenedor para la app web
- 1 contenedor para la base de datos

- 1 contenedor para el balanceador de carga

Con **Docker Compose** puedes levantarlos fácilmente en tu máquina... Pero, ¿qué pasa si necesitas:

- 100 copias de tu aplicación web (para miles de usuarios),
- que se reinicien automáticamente si fallan,
- y que todo se reparta entre varios servidores?

Ahí entra Kubernetes.

Kubernetes se encarga de:

- Crear, eliminar y reiniciar contenedores automáticamente.
- Repartirlos entre distintas máquinas (nodos).
- Escalar hacia arriba o abajo según la carga.
- Actualizar versiones sin interrumpir el servicio.
- Asignar IPs, balancear tráfico, gestionar volúmenes, etc.

Kubernetes en la nube

Cada nube tiene su propia versión administrada de Kubernetes:

Proveedor	Servicio
AWS	EKS (Elastic Kubernetes Service)
Azure	AKS (Azure Kubernetes Service)
Google Cloud	GKE (Google Kubernetes Engine)

Estos servicios te permiten usar Kubernetes **sin tener que instalarlo tú** en servidores.

Kubernetes = gestor automático de contenedores.

Permite ejecutar aplicaciones distribuidas, seguras y escalables sin preocuparte por qué servidor las ejecuta.

Diferencia entre Docker, Docker Compose, Fargate y Kubernetes

Tecnología	Qué hace	Dónde se usa
Docker	Crea y ejecuta contenedores individuales	Local o servidor
Docker Compose	Orquesta varios contenedores en una máquina	Local o pruebas
AWS Fargate	Ejecuta contenedores en la nube sin servidores	AWS (gestionado)
Kubernetes	Orquesta contenedores en muchos servidores	Local, nube o híbrido

Diferencia clave

Concepto	Qué hace	Equivalencia
Kubernetes (K8s)	Orquesta contenedores: gestiona pods, despliegues, escalado, balanceo de carga, reinicios automáticos, etc.	Plataforma completa de orquestación, muy flexible y compleja
AWS Fargate	Ejecuta contenedores sin que tú tengas que gestionar servidores	Motor de ejecución sin servidor , pero no ofrece toda la orquestación avanzada que Kubernetes hace por sí solo

Fargate es un motor de ejecución sin servidor, Kubernetes es una plataforma completa de orquestación de contenedores. |

PROBLEMAS ENCONTRADOS

no podemos usar Clusters ni Fargate en AWS Academy

Entendiendo la limitación: por qué **NO** podemos usar ECS ni Fargate en AWS Academy

Cuando trabajas en **AWS Academy**, estás **dentro de una cuenta temporal** proporcionada por el entorno educativo (no una cuenta personal de AWS).

Esa cuenta tiene **permisos restringidos**, definidos por el programa académico, para evitar costes y mantener la seguridad del entorno. Por eso, **algunos servicios de AWS no están habilitados**, y **ECS/Fargate es uno de ellos**.

¿Qué ocurre al intentar usar ECS o Fargate?

Si intentas crear un **cluster ECS** o lanzar tareas Fargate, el laboratorio te devuelve errores como:

```
User is not authorized to perform: ecs:CreateCluster
```

o

```
Fargate service is not available in this account
```

Esto sucede porque:

- **ECS (Elastic Container Service)** y **Fargate** implican **recursos de red y cómputo gestionados por AWS** (subnets, roles, VPC, instancias virtuales, etc.).
- Esos recursos **tienen coste**, y AWS Academy evita que se creen para mantener los laboratorios gratuitos.

- Las **credenciales temporales** de los laboratorios **no tienen permisos IAM** para crear roles ni clusters, algo necesario para ECS/Fargate.

Entonces... ¿qué es un *Cluster*?

Un **cluster en AWS ECS** es un **agrupador lógico** de tareas y servicios que ejecutan contenedores.

Puedes imaginarlo como un "grupo de servidores (o recursos virtuales)" donde ECS coloca y gestiona tus contenedores.

Concepto	Descripción
Cluster ECS	Es el "entorno" donde se ejecutan tus contenedores.
Task Definition	Es la "plantilla" que define cómo ejecutar cada contenedor (imagen, memoria, puertos...).
Service	Es la configuración que le dice a ECS cuántas copias (réplicas) mantener activas.
Fargate	Es el <i>modo de ejecución</i> que permite correr contenedores sin EC2: AWS gestiona toda la infraestructura.

Por qué no puedes crear un cluster ECS en AWS Academy

Motivo	Explicación
Permisos limitados	Los roles de laboratorio (LabRole) no incluyen las políticas necesarias (ecs:* , iam:* , ec2:*) para crear clusters ni roles de ejecución.
Recursos restringidos	Fargate requiere subredes, VPCs, y balanceadores de red que generan coste real, por eso AWS Academy los bloquea.
Cuentas temporales	La cuenta se destruye al final del laboratorio, por lo que los clusters, servicios y tareas desaparecerían.
Enfoque educativo	Los labs están diseñados para enseñar los conceptos, no para ejecutar servicios completos en la nube.

Simulación de despliegue Fargate

- Crear un **Cluster ECS** → **Fargate**.
- Crear una **Task Definition** apuntando a la imagen en ECR.
- Crear un **Service Fargate** con el número de réplicas deseado.
- Configurar **puertos** y **roles de ejecución** (IAM Task Role).

Esto es **teórico**, porque el Lab Role no permite desplegar Fargate directamente.

Alternativa: prácticas con **EC2 + Docker**

Aunque no podemos usar ECS ni Fargate, **sí podemos trabajar con contenedores en AWS** usando **instancias EC2**.

De esta manera, nosotros mismos gestionamos la "infraestructura" (la máquina virtual EC2), simulando lo que haría ECS/Fargate automáticamente.

- Instalar Docker en una instancia EC2

1. **Lanza una instancia EC2** desde la consola de AWS (por ejemplo, Amazon Linux 2).

2. Conéctate por SSH:

```
ssh -i "tu-clave.pem" ec2-user@<public-ip>
```

3. **Instala Docker:**

```
sudo yum update -y
sudo amazon-linux-extras install docker -y
sudo service docker start
sudo usermod -aG docker ec2-user
```

4. Verifica:

```
docker --version
docker ps
```

Con esto, tu instancia EC2 actúa como "mini-servidor" donde puedes ejecutar contenedores.

- Ejecutar contenedores y Docker Compose en EC2

Una vez tienes Docker, puedes **subir y ejecutar tus imágenes** manualmente, o incluso desplegar aplicaciones completas con Docker Compose.

1. **Sube tu imagen desde ECR (si la creaste antes):**

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS
--password-stdin <account-id>.dkr.ecr.us-east-1.amazonaws.com
docker pull <account-id>.dkr.ecr.us-east-1.amazonaws.com/mi-app:latest
```

2. **Ejecuta tu contenedor directamente:**

```
docker run -d -p 80:80 <account-id>.dkr.ecr.us-east-1.amazonaws.com/mi-
app:latest
```

3. **Instala Docker Compose** (si tu app usa varios contenedores):

```
sudo curl -L
"https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

4. **Lanza todo con Compose:**

```
docker compose up -d
```

En resumen

Concepto	En AWS Academy	Alternativa práctica
ECS (Clusters)	No permitido	Ejecuta contenedores manualmente en EC2
Fargate	No permitido	Usa Docker Compose o <code>docker run</code> en EC2
ECR (repositorio)	Permitido	Puedes crear repositorios y subir imágenes
Docker local	Permitido	Prueba tus apps antes de subirlas
Persistencia de recursos	Temporal	Guarda imágenes y código localmente

Conclusión didáctica

- En **AWS real**, usarías **ECS + Fargate** para desplegar tus contenedores sin preocuparte por servidores.
- En **AWS Academy**, **no puedes** crear clusters ni usar Fargate, porque los laboratorios:
 - usan **cuentas temporales**,
 - tienen **permisos reducidos**,
 - y no permiten **servicios de infraestructura con coste**.

Pero sí puedes **aprender exactamente los mismos conceptos** practicando con:

- **Docker Compose localmente**
- **EC2 + Docker manualmente**

Así entiendes cómo funcionan los contenedores, cómo se despliegan, y cómo se integran con AWS, **sin depender de Fargate**.

EJERCICIO A ENTREGAR

Práctica: Ejecutar tu imagen Docker en EC2 y ECR Antes de comenzar, asegúrate de haber **construido tu imagen Docker en tu máquina local** y de **subirla a un repositorio en Amazon ECR**, ya que en esta práctica la instancia EC2 deberá **descargar esa imagen desde ECR y ejecutarla como contenedor**.

- Paso 1: Crear la EC2 Consola AWS → EC2 → Launch Instance Tipo de instancia: Sistema operativo: Configura Security Group para permitir tráfico HTTP Descarga la key PEM para conectarte via SSH
- Paso 2: Conectarte a la EC2
- Paso 3: Instalar Docker `sudo yum update -y` `sudo amazon-linux-extras install docker -y` `sudo service docker start` `sudo usermod -a -G docker ec2-user`

Cierra sesión y vuelve a conectarte para que el grupo docker se aplique. Verifica Docker: `docker info`

- Paso 4: Instalar AWS CLI (si no está)

Configura AWS CLI con tus credenciales si es necesario: `aws configure`

- Paso 5: Autenticarse en ECR `aws ecr get-login-password --region | docker login --username AWS --password-stdin <AWS_ACCOUNT_ID>.dkr.ecr..amazonaws.com`
- Paso 6: Descargar la imagen de ECR `docker pull <AWS_ACCOUNT_ID>.dkr.ecr..amazonaws.com/mi_app:1.0`

Verifica que la imagen está en la EC2: `docker images`

- Paso 7: Ejecutar la imagen `docker run -d -p 5000:5000 <AWS_ACCOUNT_ID>.dkr.ecr..amazonaws.com/mi_app:1.0`

`-d` → Ejecuta en segundo plano

`-p 5000:5000` → Mapea el puerto 5000 de la EC2 al contenedor

Abre en tu navegador: `http://:5000`

Deberías ver tu mensaje de Flask: ¡Hola desde Flask en contenedor!

Entrega:

Documentar todo el proceso y subir al aula virtual